

Document number:	P0579R0
Date:	2017-01-23
Project:	C++ Extensions for Ranges, Library Working Group
Reply-to:	Casey Carter < Casey@Carter.net >

Constexpr for <experimental/ranges/iterator>

- [1 Synopsis](#)
- [2 Background](#)
 - [2.1 Design issues: common_iterator](#)
 - [2.2 Implementation Experience](#)
- [3 Proposed Resolution](#)

1 Synopsis

This paper proposes changes to the specification of components declared in the Ranges TS header <experimental/ranges/iterator> to enable them to be used in a constexpr context. These changes resolve Ranges TS issue 256 “Add constexpr to advance, distance, next and prev” (<https://github.com/ericniebler/stl2/issues/256>), and issue 322 “ranges::exchange should be constexpr and conditionally noexcept” (<https://github.com/ericniebler/stl2/issues/322>).

2 Background

During review of P0370R2 “Ranges TS Design Updates Omnibus” at Issaquah, LWG directed that functions declared in the header <experimental/ranges/iterator> should be constexpr if they are so in the C++ WP header <iterator>. It was suggested that the new iterator adaptors added in the Ranges TS, counted_iterator and common_iterator, should also be “as constexpr as possible” in the spirit of e.g. reverse_iterator and move_iterator. We’ve enlarged the scope slightly to include some utilities (tagged, dangling, and exchange) to enable the use of the iterator tools to implement constexpr algorithms.

This paper proposes changes to the specification of:

- the iterator operations advance, next, prev, and distance
- the iterator adaptors reverse_iterator, move_iterator, counted_iterator, and common_iterator
- the sentinel adaptor move_sentinel
- the dangling wrapper class template
- class template tagged
- function template exchange

to enable those components to be used in the implementation of constexpr algorithms.

2.1 Design issues: common_iterator

The design intent of common_iterator<I, S>, for some iterator type I and sentinel type S, is that it be implementable in terms of a C++17 std::variant<I, S>. Unfortunately the limitations of constexpr in C++17 make it impossible to implement the assignment operators of std::variant so as to be generally usable in a constexpr context. Consequently, common_iterator’s assignment operators cannot in general be usable in constexpr context.

Similarly, if either I or S has a nontrivial destructor, common_iterator<I, S> must also have a nontrivial destructor, disqualifying it from being a literal type.

Since an iterator that cannot be assigned is of extremely limited utility, we do not propose to apply constexpr to all member functions of common_iterator at this time; we propose only that the constructors of common_iterator be constexpr.

2.2 Implementation Experience

All of the suggested changes have been implemented in CMCSTL2 (<https://github.com/caseycarter/cmcstl2>) as a sanity check.

3 Proposed Resolution

Change the synopsis of header <experimental/ranges/utility> in [utility]/2 as follows:

```
// 5.2.1, swap:
namespace {
    constexpr unspecified swap = unspecified;
}

// 5.2.2, exchange:
template <MoveConstructible T, class U=T>
    requires Assignable<T&, U>()
constexpr T exchange(T& obj, U&& new_val) noexcept(see below);

// 5.5.2, struct with named accessors
template <class T>
concept bool TagSpecifier() {
    return see below;
}
```

Change the declaration of exchange in [utility.exchange] to agree, and add a new paragraph after paragraph 1:

```
template <MoveConstructible T, class U=T>
    requires Assignable<T&, U>()
constexpr T exchange(T& obj, U&& new_val) noexcept(see below);
```

1 *Effects*: Equivalent to:

```
T old_val = std::move(obj);
obj = std::forward<U>(new_val);
return old_val;
```

2 *Remarks*: The expression in the noexcept is equivalent to:

```
is_nothrow_move_constructible<T>::value &&
is_nothrow_assignable<T&, U>::value
```

Change the synopsis of class templated tagged in [taggedup.tagged]/2 as follows:

```
template <class Base, TagSpecifier... Tags>
    requires sizeof...(Tags) <= tuple_size<Base>::value
struct tagged :
    Base, TAGGET(tagged<Base, Tags...>, Ti, i)... { // see below
    using Base::Base;
    tagged() = default;
    tagged(tagged&&) = default;
    tagged(const tagged&) = default;
    tagged& operator=(tagged&&) = default;
    tagged& operator=(const tagged&) = default;
    template <class Other>
        requires Constructible<Base, Other>()
    constexpr tagged(tagged<Other, Tags...>&& that) noexcept(see below);
    template <class Other>
        requires Constructible()
    constexpr tagged(const tagged<Other, Tags...>& that);
    template <class Other>
        requires Assignable<Base&, Other>()
    constexpr tagged& operator=(tagged<Other, Tags...>&& that) noexcept(see below);
    template <class Other>
        requires Assignable<Base&, const Other&>()
    constexpr tagged& operator=(const tagged<Other, Tags...>& that);
    template <class U>
        requires Assignable<Base&, U>() && !Same<decay_t<U>, tagged>()
    constexpr tagged& operator=(U&& u) noexcept(see below);
    constexpr void swap(tagged& that) noexcept(see below)
        requires Swappable<Base&>();
    constexpr friend void swap(tagged&, tagged&) noexcept(see below)
        requires Swappable<Base&>();
};
```

Update the specifications of the functions in [taggedup.tagged] to agree with the synopsis.

Change the synopsis of the header <experimental/ranges/iterator> in [iterator.synopsis] as follows:

```
// 6.6.5, iterator operations:
template <Iterator I>
    constexpr void advance(I& i, difference_type_t<I> n);
template <Iterator I, Sentinel<I> S>
    constexpr void advance(I& i, S bound);
template <Iterator I, Sentinel<I> S>
    constexpr difference_type_t<I> advance(I& i, difference_type_t<I> n, S bound);
template <Iterator I, Sentinel<I> S>
```

```

constexpr difference_type_t<I> distance(I first, S last);
template <Iterator I>
constexpr I next(I x, difference_type_t<I> n = 1);
template <Iterator I, Sentinel<I> S>
constexpr I next(I x, S bound);
template <Iterator I, Sentinel<I> S>
constexpr I next(I x, difference_type_t<I> n, S bound);
template <BidirectionalIterator I>
constexpr I prev(I x, difference_type_t<I> n = 1);
template <BidirectionalIterator I>
constexpr I prev(I x, difference_type_t<I> n, I bound);

```

[...]

// 6.7, predefined iterators and sentinels:

// 6.7.1, reverse iterators:

```

template <BidirectionalIterator I> class reverse_iterator;

```

```

template <class I1, class I2>
    requires EqualityComparable<I1, I2>()
constexpr bool operator==(
    const reverse_iterator<I1>& x,
    const reverse_iterator<I2>& y);

```

```

template <class I1, class I2>
    requires EqualityComparable<I1, I2>()
constexpr bool operator!=(
    const reverse_iterator<I1>& x,
    const reverse_iterator<I2>& y);

```

```

template <class I1, class I2>
    requires StrictTotallyOrdered<I1, I2>()
constexpr bool operator<(
    const reverse_iterator<I1>& x,
    const reverse_iterator<I2>& y);

```

```

template <class I1, class I2>
    requires StrictTotallyOrdered<I1, I2>()
constexpr bool operator>(
    const reverse_iterator<I1>& x,
    const reverse_iterator<I2>& y);

```

```

template <class I1, class I2>
    requires StrictTotallyOrdered<I1, I2>()
constexpr bool operator>=(
    const reverse_iterator<I1>& x,
    const reverse_iterator<I2>& y);

```

```

template <class I1, class I2>
    requires StrictTotallyOrdered<I1, I2>()
constexpr bool operator<=(
    const reverse_iterator<I1>& x,
    const reverse_iterator<I2>& y);

```

```

template <class I1, class I2>
    requires SizedSentinel<I1, I2>()
constexpr difference_type_t<I2> operator-(
    const reverse_iterator<I1>& x,
    const reverse_iterator<I2>& y);

```

```

template <RandomAccessIterator I>
constexpr reverse_iterator<I> operator+(
    difference_type_t<I> n,
    const reverse_iterator<I>& x);

```

```

template <BidirectionalIterator I>
constexpr reverse_iterator<I> make_reverse_iterator(I i);

```

[...]

// 6.7.3, move iterators and sentinels:

```

template <InputIterator I> class move_iterator;

```

```

template <class I1, class I2>
    requires EqualityComparable<I1, I2>()
constexpr bool operator==(
    const move_iterator<I1>& x, const move_iterator<I2>& y);

```

```

template <class I1, class I2>
    requires EqualityComparable<I1, I2>()
constexpr bool operator!=(
    const move_iterator<I1>& x, const move_iterator<I2>& y);

```

```

template <class I1, class I2>
    requires StrictTotallyOrdered<I1, I2>()
constexpr bool operator<(

```

```

    const move_iterator<I1>& x, const move_iterator<I2>& y);
template <class I1, class I2>
    requires StrictTotallyOrdered<I1, I2>()
    constexpr bool operator==(
        const move_iterator<I1>& x, const move_iterator<I2>& y);
template <class I1, class I2>
    requires StrictTotallyOrdered<I1, I2>()
    constexpr bool operator>(
        const move_iterator<I1>& x, const move_iterator<I2>& y);
template <class I1, class I2>
    requires StrictTotallyOrdered<I1, I2>()
    constexpr bool operator>=(
        const move_iterator<I1>& x, const move_iterator<I2>& y);

template <class I1, class I2>
    requires SizedSentinel<I1, I2>()
    constexpr difference_type_t<I2> operator-(
        const move_iterator<I1>& x,
        const move_iterator<I2>& y);
template <RandomAccessIterator I>
    constexpr move_iterator<I> operator+(
        difference_type_t<I> n,
        const move_iterator<I>& x);
template <InputIterator I>
    constexpr move_iterator<I> make_move_iterator(I i);

```

```

template <Semiregular S> class move_sentinel;

```

```

template <class I, Sentinel<I> S>
    constexpr bool operator==(
        const move_iterator<I>& i,
        const move_sentinel<S>& s);
template <class I, Sentinel<I> S>
    constexpr bool operator==(
        const move_sentinel<S>& s,
        const move_iterator<I>& i);
template <class I, Sentinel<I> S>
    constexpr bool operator!=(
        const move_iterator<I>& i,
        const move_sentinel<S>& s);
template <class I, Sentinel<I> S>
    constexpr bool operator!=(
        const move_sentinel<S>& s,
        const move_iterator<I>& i);

template <class I, SizedSentinel<I> S>
    constexpr difference_type_t<I> operator-(
        const move_sentinel<S>& s, const move_iterator<I>& i);
template <class I, SizedSentinel<I> S>
    constexpr difference_type_t<I> operator-(
        const move_iterator<I>& i, const move_sentinel<S>& s);

```

```

template <Semiregular S>
    constexpr move_sentinel<S> make_move_sentinel(S s);

```

[...]

// 6.7.6, counted iterators:

```

template <Iterator I> class counted_iterator;

template <class I1, class I2>
    requires Common<I1, I2>()
    constexpr bool operator==(
        const counted_iterator<I1>& x, const counted_iterator<I2>& y);
constexpr bool operator==(
    const counted_iterator<auto>& x, default_sentinel);
constexpr bool operator==(
    default_sentinel, const counted_iterator<auto>& x);
template <class I1, class I2>
    requires Common<I1, I2>()
    constexpr bool operator!=(
        const counted_iterator<I1>& x, const counted_iterator<I2>& y);
constexpr bool operator!=(
    const counted_iterator<auto>& x, default_sentinel y);
constexpr bool operator!=(
    default_sentinel x, const counted_iterator<auto>& y);
template <class I1, class I2>
    requires Common<I1, I2>()
    constexpr bool operator<

```

```

    const counted_iterator<I1>& x, const counted_iterator<I2>& y);
template <class I1, class I2>
    requires Common<I1, I2>()
    constexpr bool operator<=(
        const counted_iterator<I1>& x, const counted_iterator<I2>& y);
template <class I1, class I2>
    requires Common<I1, I2>()
    constexpr bool operator>(
        const counted_iterator<I1>& x, const counted_iterator<I2>& y);
template <class I1, class I2>
    requires Common<I1, I2>()
    constexpr bool operator>=(
        const counted_iterator<I1>& x, const counted_iterator<I2>& y);
template <class I1, class I2>
    requires Common<I1, I2>()
    constexpr difference_type_t<I2> operator-(
        const counted_iterator<I1>& x, const counted_iterator<I2>& y);
template <class I>
    constexpr difference_type_t<I> operator-(
        const counted_iterator<I>& x, default_sentinel y);
template <class I>
    constexpr difference_type_t<I> operator-(
        default_sentinel x, const counted_iterator<I>& y);

template <RandomAccessIterator I>
    constexpr counted_iterator<I> operator+(
        difference_type_t<I> n, const counted_iterator<I>& x);
template <Iterator I>
    constexpr counted_iterator<I> make_counted_iterator(I i, difference_type_t<I> n);
template <Iterator I>
    constexpr void advance(counted_iterator<I>& i, difference_type_t<I> n);

[...]
```

```

// 6.11, range primitives:
namespace {
    constexpr unspecified size = unspecified;
    constexpr unspecified empty = unspecified;
    constexpr unspecified data = unspecified;
    constexpr unspecified cdata = unspecified;
}
template <Range R>
    constexpr difference_type_t<iterator_t<R>> distance(R&& r);
template <SizedRange R>
    constexpr difference_type_t<iterator_t<R>> distance(R&& r);

```

Update the specifications of the functions in [iterator.operations], [reverse.iterator], [iterators.move], [move.sentinel], [iterators.counted], and [range.primitives] to agree with the synopsis.

In [reverse.iterator], change the synopsis of class template `reverse_iterator` as follows:

```

template <BidirectionalIterator I>
class reverse_iterator {
public:
    using iterator_type = I;
    using difference_type = difference_type_t<I>;
    using value_type = value_type_t<I>;
    using iterator_category = iterator_category_t<I>;
    using reference = reference_t<I>;
    using pointer = I;

    constexpr reverse_iterator();
    constexpr explicit reverse_iterator(I x);
    constexpr reverse_iterator(const reverse_iterator<ConvertibleTo<I>>& i);
    constexpr reverse_iterator& operator=(const reverse_iterator<ConvertibleTo<I>>& i);

    constexpr I base() const;
    constexpr reference operator*() const;
    constexpr pointer operator->() const;

    constexpr reverse_iterator& operator++();
    constexpr reverse_iterator operator++(int);
    constexpr reverse_iterator& operator--();
    constexpr reverse_iterator operator--(int);

    constexpr reverse_iterator operator+ (difference_type n) const
        requires RandomAccessIterator<I>();
    constexpr reverse_iterator& operator+=(difference_type n)
        requires RandomAccessIterator<I>();

```

```

constexpr reverse_iterator operator- (difference_type n) const
    requires RandomAccessIterator<I>();
constexpr reverse_iterator& operator-=(difference_type n)
    requires RandomAccessIterator<I>();
constexpr reference operator const
    requires RandomAccessIterator<I>();
private:
    I current; // exposition only
};

```

Update the specifications of the member functions in [reverse.iterator] to agree with the class template synopsis.

In [move.iterator], change the synopsis of class template `move_iterator` as follows:

```

template <InputIterator I>
class move_iterator {
public:
    using iterator_type = I;
    using difference_type = difference_type_t<I>;
    using value_type = value_type_t<I>;
    using iterator_category = input_iterator_tag;
    using reference = see below;

    move_iterator();
    constexpr explicit move_iterator(I i);
    constexpr move_iterator(const move_iterator<ConvertibleTo<I>>& i);
    constexpr move_iterator& operator=(const move_iterator<ConvertibleTo<I>>& i);

    constexpr I base() const;
    constexpr reference operator*() const;

    constexpr move_iterator& operator++();
    constexpr move_iterator operator++(int);
    constexpr move_iterator& operator-()
        requires BidirectionalIterator<I>();
    constexpr move_iterator operator-(int)
        requires BidirectionalIterator<I>();

    constexpr move_iterator operator+(difference_type n) const
        requires RandomAccessIterator<I>();
    constexpr move_iterator& operator+=(difference_type n)
        requires RandomAccessIterator<I>();
    constexpr move_iterator operator-(difference_type n) const
        requires RandomAccessIterator<I>();
    constexpr move_iterator& operator-=(difference_type n)
        requires RandomAccessIterator<I>();
    constexpr reference operator const
        requires RandomAccessIterator<I>();

private:
    I current; // exposition only
};

```

Update the specifications of the member functions in [move.iterator] to agree with the class template synopsis.

In [common.iterator], change the synopsis of class template `common_iterator` as follows:

```

template <Iterator I, Sentinel<I> S>
    requires !Same<I, S>()
class common_iterator {
public:
    using difference_type = difference_type_t<I>;

    constexpr common_iterator();
    constexpr common_iterator(I i);
    constexpr common_iterator(S s);
    constexpr common_iterator(const common_iterator<ConvertibleTo<I>, ConvertibleTo<S>>& u);
    common_iterator& operator=(const common_iterator<ConvertibleTo<I>, ConvertibleTo<S>>& u);

    ~common_iterator();

    see below operator*();
    see below operator*() const;
    see below operator->() const requires Readable<I>();

    common_iterator& operator++();
    common_iterator operator++(int);

private:

```

```

    bool is_sentinel; // exposition only
    I iter; // exposition only
    S sentinel; // exposition only
};

```

Update the specifications of the member functions in [common.iterator] to agree with the class template synopsis.

In [counted.iterator], change the synopsis of class template `counted_iterator` as follows:

```

template <Iterator I>
class counted_iterator {
public:
    using iterator_type = I;
    using difference_type = difference_type_t<I>;

    constexpr counted_iterator();
    constexpr counted_iterator(I x, difference_type_t<I> n);
    constexpr counted_iterator(const counted_iterator<ConvertibleTo<I>>& i);
    constexpr counted_iterator& operator=(const counted_iterator<ConvertibleTo<I>>& i);

    constexpr I base() const;
    constexpr difference_type_t<I> count() const;
    constexpr see below operator*();
    constexpr see below operator*() const;

    constexpr counted_iterator& operator++();
    constexpr counted_iterator operator++(int);
    constexpr counted_iterator& operator--()
        requires BidirectionalIterator<I>();
    constexpr counted_iterator operator--(int)
        requires BidirectionalIterator<I>();

    constexpr counted_iterator operator+ (difference_type n) const
        requires RandomAccessIterator<I>();
    constexpr counted_iterator& operator+=(difference_type n)
        requires RandomAccessIterator<I>();
    constexpr counted_iterator operator- (difference_type n) const
        requires RandomAccessIterator<I>();
    constexpr counted_iterator& operator-=(difference_type n)
        requires RandomAccessIterator<I>();
    constexpr see below operator const
        requires RandomAccessIterator<I>();
private:
    I current; // exposition only
    difference_type_t<I> cnt; // exposition only
};

```

Update the specifications of the member functions in [counted.iterator] to agree with the class template synopsis.

In [dangling.wrap], change the synopsis of class template `dangling` as follows:

```

template <CopyConstructible T>
class dangling {
public:
    constexpr dangling() requires DefaultConstructible<T>();
    constexpr dangling(T t);
    constexpr T get_unsafe() const;
private:
    T value; // exposition only
};

```

Update the specifications of the member functions in [dangling.wrap.ops] to agree with the class template synopsis.